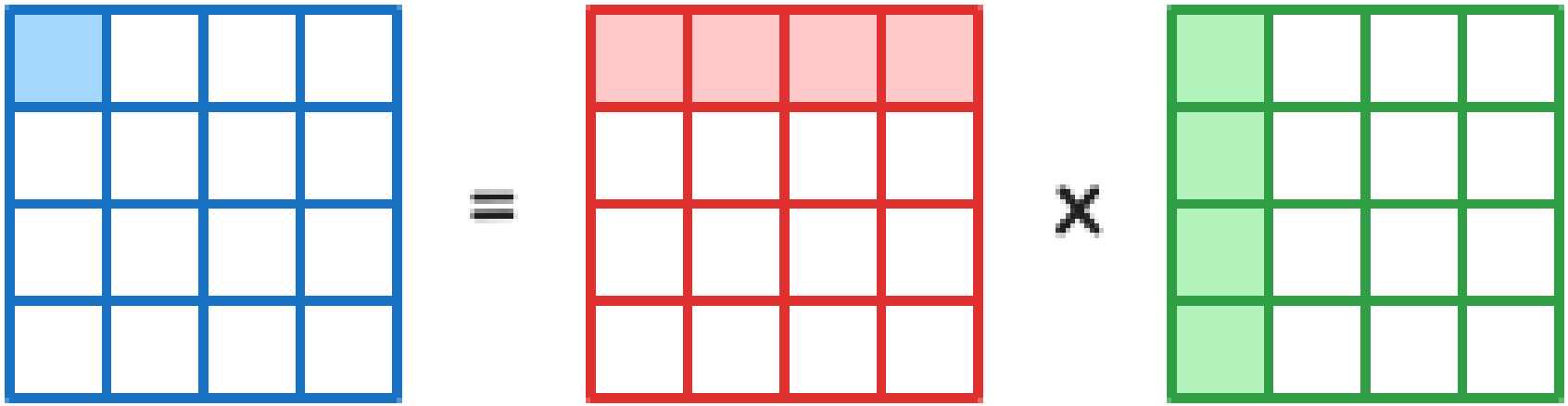
Table of Contents

1 Why GPUs? What's wrong with CPUs?

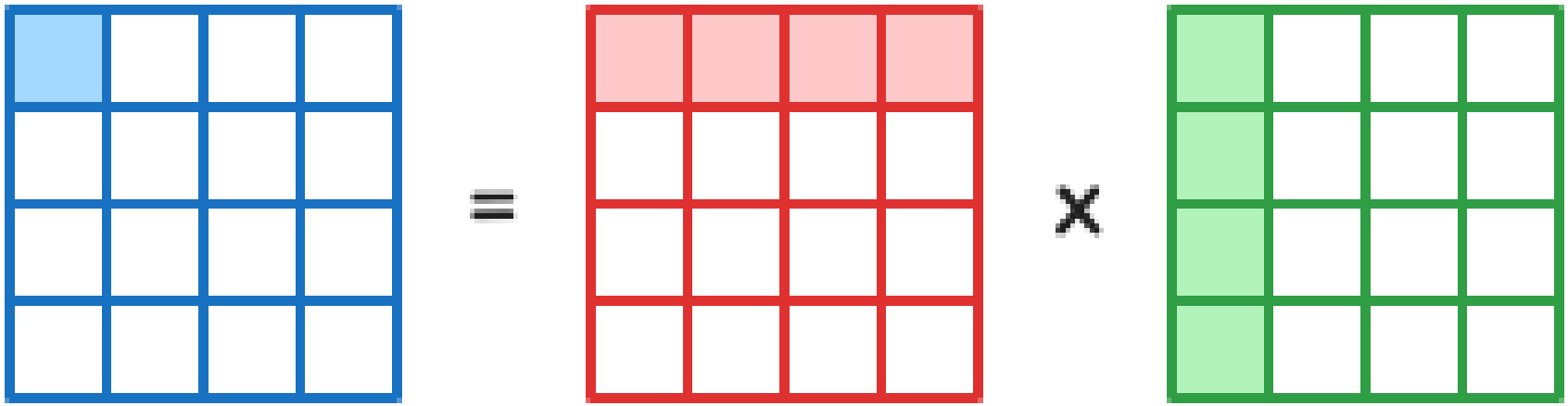
Computing a Matrix Multiplication

Let A and B be two 4×4 matrices. The matrix multiple $C = AB$ results in another 4×4 matrix.



Computing a Matrix Multiplication

Let A and B be two 4×4 matrices. The matrix multiple $C = AB$ results in another 4×4 matrix.



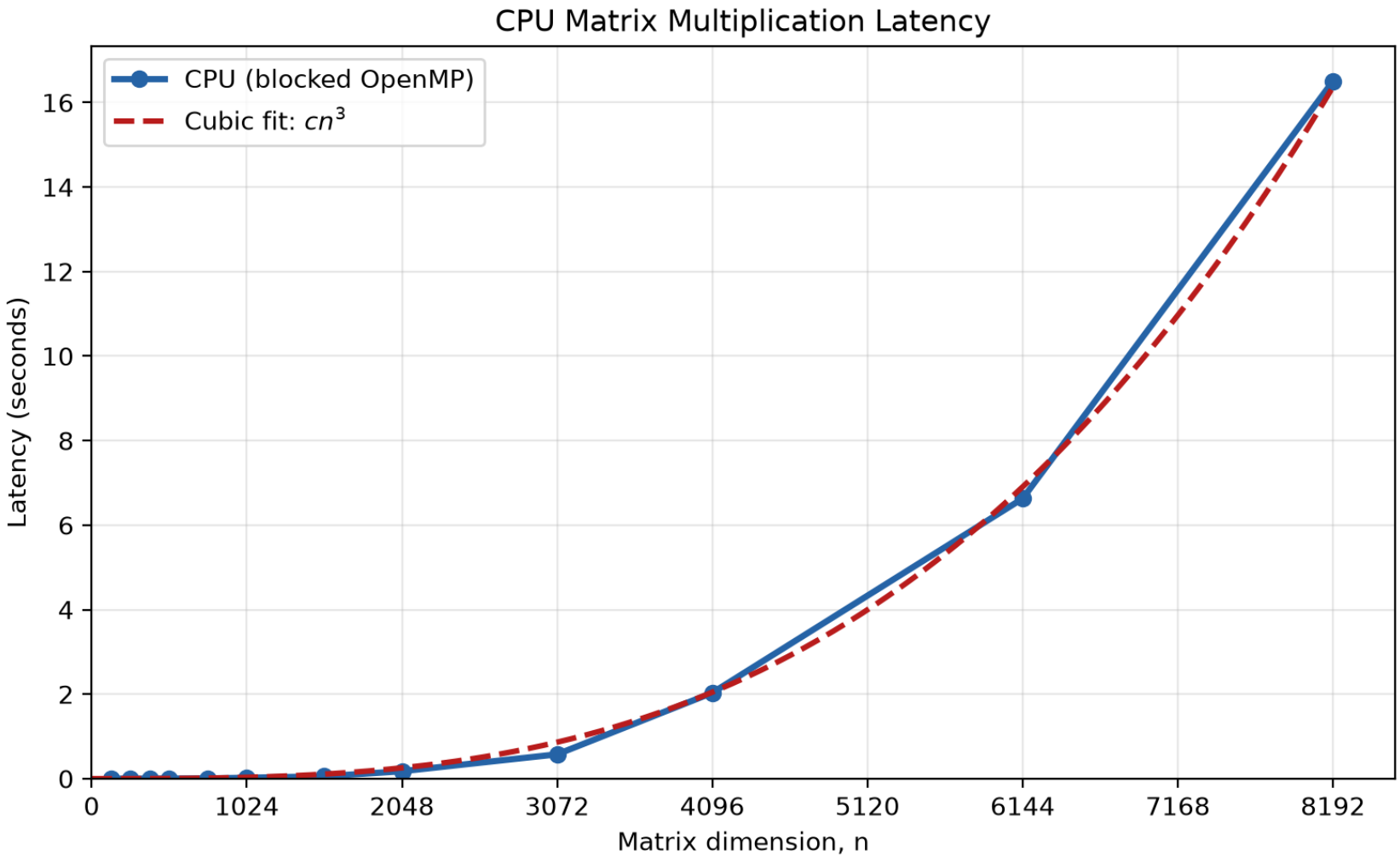
Poor scaling

The operations are very simple, but the number of operations grow on the order of $\mathcal{O}(n^3)$, where n is the dimensions of the matrices.

Runtime of a matrix multiplication on a CPU

```
for (std::size_t y = 0; y < n; ++y) {
  for (std::size_t x = 0; x < n; ++x) {
    double sum = 0.0;
    for (std::size_t k = 0; k < n; ++k) {
      sum += a[y*n+k] * b[k*n+x];
    }
    c[y*n+x] = sum;
  }
}
```

Runtime of a matrix multiplication on a CPU



What are GPUs for?

Real-time 3D graphics requires the same kinds of operations to be repeated across millions of vertices and pixels.

What are GPUs for?

Real-time 3D graphics requires the same kinds of operations to be repeated across millions of vertices and pixels.

Graphics exposes data parallelism

Most vertices and pixels can be processed independently. The GPU can run the same operations on many different pieces of data at once.

What are GPUs for?

Real-time 3D graphics requires the same kinds of operations to be repeated across millions of vertices and pixels.

Graphics exposes data parallelism

Most vertices and pixels can be processed independently. The GPU can run the same operations on many different pieces of data at once.

A CPU uses a small number of sophisticated cores to minimize the latency of one instruction stream. A GPU instead uses many simpler execution units to maximize *throughput*.

What are GPUs for?

Real-time 3D graphics requires the same kinds of operations to be repeated across millions of vertices and pixels.

Graphics exposes data parallelism

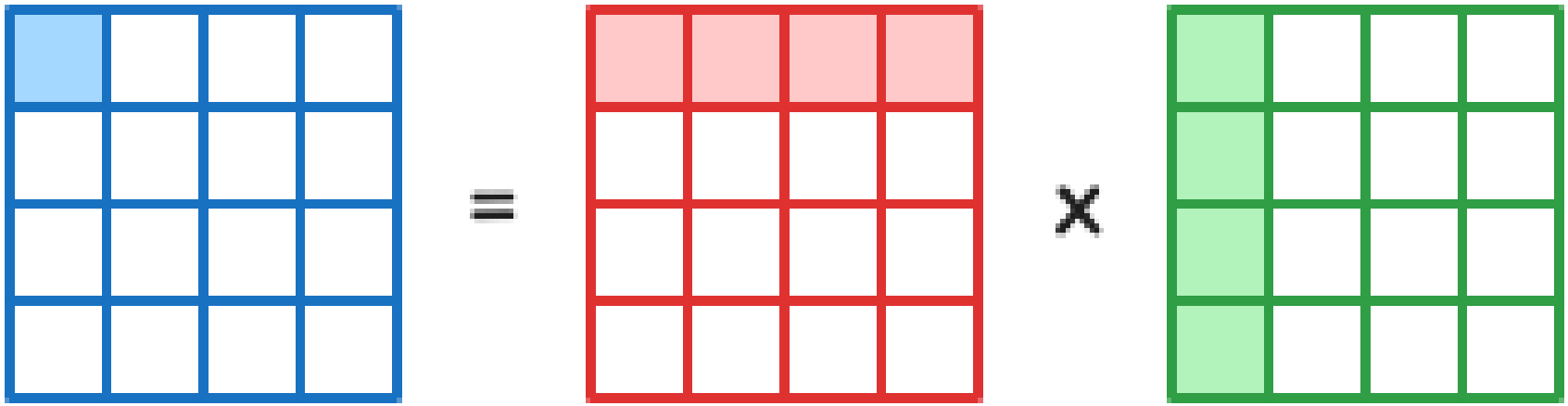
Most vertices and pixels can be processed independently. The GPU can run the same operations on many different pieces of data at once.

A CPU uses a small number of sophisticated cores to minimize the latency of one instruction stream. A GPU instead uses many simpler execution units to maximize *throughput*.

The key tradeoff

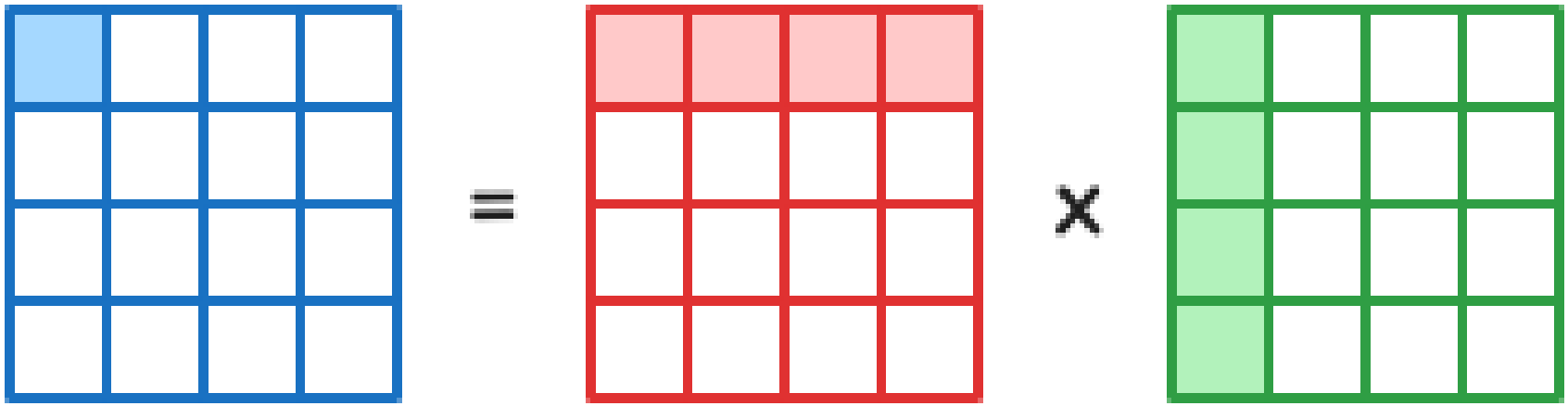
Graphics rendering is difficult because of the **volume** of similar computations, not because any one computation is especially complex.

Matrix Multiplication is Parallel



Each element of the output is independent. The results of C_{11} does not effect the results of C_{12}, or any other element.

Matrix Multiplication is Parallel



Each element of the output is independent. The results of C_{11} does not effect the results of C_{12}, or any other element.

Key Idea

If matrix multiplication is a **parallel** task, and GPU's were designed for the **parallel** task of rendering graphics, can we repurpose GPU's to do matrix multiplication?

Takeaway

- CPUs can't efficiently perform matrix multiplication at scale.
- Graphics rendering is a parallel task, and so is matrix multiplication. Thus, GPUs can be good at matrix multiplication.
- GEMM is a generalization of of matrix multiplication, and is the operation we will be discussing.

Whats next

I've suggested that GPUs *can* do matrix multiplication, but how do we actually get there?